

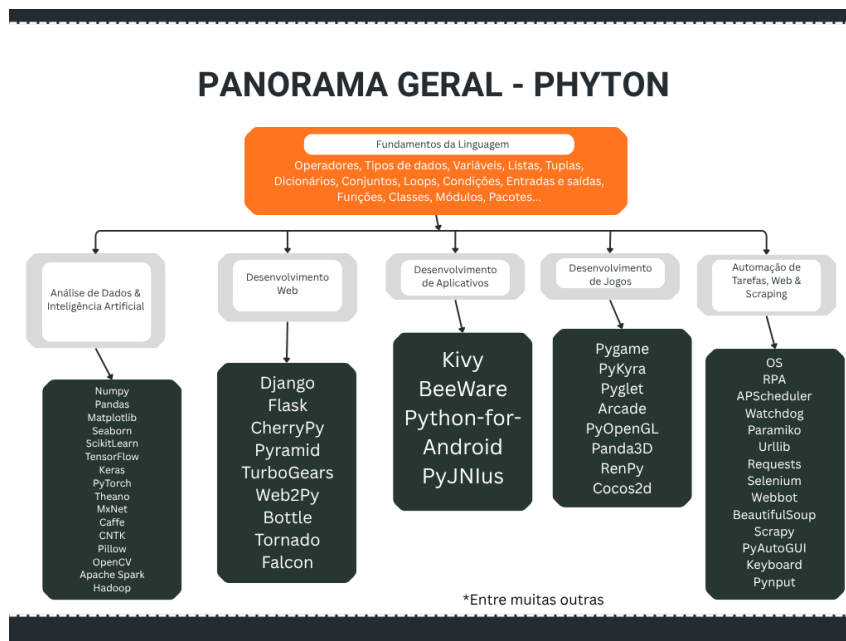
Relatório 02 - Prática: Linguagem de Programação Python (I)

Gabriel Oliveira Santos

1. Introdução

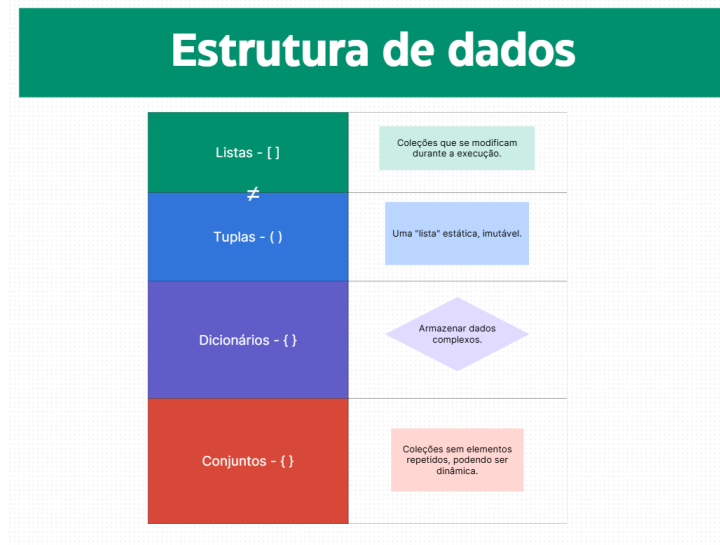
O presente relatório tem o intuito de consolidar todo o conhecimento adquirido no minicurso de Python, transformando em algo tangível e de fácil entendimento. Foram abordados desde conteúdos básicos até funções um pouco avançadas. Essa experiência foi essencial para aprofundar meu conhecimento em uma linguagem na qual eu tinha pouca prática.

2. Desenvolvimento



O curso pode me apresentar muitas coisas que haviam passado despercebido em meus conhecimentos sobre os fundamentos da linguagem Python. Tudo o que foi apresentado ali, dividi em tópicos, incluindo detalhes, entretanto é válido ressaltar que foram passadas de forma rápida no curso, mas muito eficiente para quem já teve algum contato com programação. Segue meu resumo:

- Estruturas de dados:



- Listas: Estrutura simples de dado, usadas para armazenar coleções dinâmicas, sendo definida pelos colchetes.
- Tuplas: Forma de armazenamento de valores, quase uma lista, mas imutável, estática, definida pelos parênteses.
- Dicionários: A forma mais abrangente diante das anteriores, permite múltiplas características dentro de si, o que é essencial pra armazenar dados complexos, essa é definida pelas chaves.
- Conjuntos: Assim como na matemática, são também definidos também por chaves, mas podem ser também definidos por uma função set(). Importante*, eles eliminam elementos redundantes.

- Funções



- Funções tradicionais:
 - *if* - se, condicional.

- *else* – Se não, condicional.
 - *elif* – Se/Ou também - que é o else if, mas abreviado, condicional.
 - *for* – de – Laço de repetição controlado, mediante as iterações.
 - *while* – enquanto – Laço de repetição que não tem fim enquanto não for satisfeita a condição estabelecida.
 - *def* – *definition* – É basicamente a definição de uma função, que pode ser reutilizada posteriormente.
- Funções anônimas:
 - Lambda: Serve principalmente para simplificar os códigos, ao invés de fazer uma função gigante, ou com métodos e funções tradicionais, essa consegue manter em apenas uma linha, mas tem seu defeito de ser “descartável”.
 - Reduce: Serve literalmente para reduzir a carga de outras funções, “concatenando” duas ou mais funções, a fim de obter uma só resposta. Útil para operações que acumulam ou condensam dados.
- **Funções que me chamaram a atenção:**
 - Map, filter, reduce, lambda
 - Map serve para mapear e enumerar uma lista, filter para filtrar pelo atributo desejado (O que facilita absurdamente alguns programas), reduce e lambda já foram citadas acima.
 - Todas aumentam a flexibilidade e tornam o código mais expressivo, mas para um iniciante, pode dificultar a leitura de algumas dessas funções.
- **Pacotes e Módulos:**
 - Importação: As importações de outros arquivos são importantes para rodar funções externas ao seu atual arquivo, usa-se *import <diretório.arquivo>* ou *from <diretório> import <arquivo(s)>*
 - Pacotes: O Diretório.
 - Módulos: O arquivo.
 - Para identificar onde está sendo executado, respectivamente, `__package__` e `__name__` .

- Orientação a objetos

Nome da Classe (Ex: Pessoa)
Atributos: nome, cpf, altura, data de nascimento...
Métodos: Falar(), Comer(), Correr(), Andar(), Correr()...

- Criação de classes: Para criar uma classe, percebi que é extremamente simples na linguagem, basta chamar `class <Nome da Classe>` e incluir os dados desejados.
 - Atributos e métodos: Aqui é o que vai dentro da classe, pois pode ter atributos visíveis a outros arquivos (Público) ou invisíveis (Privados – não existem “privados” de verdade, a convenção é por `[__atributo]`). Os métodos por sua vez são funções dentro da classe. Aqui entram os getters e setters, funções que servem para serem chamadas por outros arquivos, a fim de não mexer diretamente na classe. Eles são criados por `@property` e `@atributo.setter`, dependendo do propósito.
 - Herança e sobrescrita com `super()`
 - Métodos de classe (`@classmethod`) e estáticos (`@staticmethod`)
- Operadores
 - Unários (`-`, `not`), binários (`+`, `-`, `*`, `/`, `%`), relacionais (`==`, `!=`, `<`, `>`), atribuição (`+=`, `-=`), lógicos (`and`, `or`, `not`), ternário (`x if cond else y`).

No fim, tudo isso serve para organizar o código em moldes reutilizáveis e escaláveis, o que foi um aprendizado bem útil e interessante, já que a linguagem não é tipada/verbosa como o Java. Os únicos pontos que tive de estar atento, foi em relação aos espaçamentos, para estruturar os blocos de código corretamente.

Boas práticas de código foram vistas, mas nada tão aprofundado, já que o intuito do vídeo era uma introdução a linguagem.

3. Conclusão

O estudo do Python nesse minicurso mostrou não apenas a sintaxe da linguagem, mas também o poder e a versatilidade dela, capaz de atender quase todos os tipos de tarefa. A tipagem dinâmica foi algo estranho no início, mas prática no uso. Aprender funções como `map`, `filter` e `lambda` foi divertido e mostrou como simplificar

operações complexas em poucas linhas. Estruturas como listas, tuplas... reforçaram a importância de escolher bem a forma de armazenar dados. Pacotes e módulos ajudaram a organizar o código, e a orientação a objetos trouxe clareza sobre como criar moldes reutilizáveis e escaláveis. A principal conclusão é que Python, mesmo para iniciantes, já se mostra robusto e pronto para aplicações mais sofisticadas.